Optimization of the step-model likelihood search: from $O(n \times m)$ to $O(k \times m)$

Package: `friends.test`
Branch: `biocparallel-opt`
File: `R/step.fit.ln.likelihoods.r`, `R/best.step.fit.r`, `R/best.step.fit.bic.r`

---

1. Problem setting

Let A be a real association matrix of shape $m \times n$, where m rows are candidate markers (e.g. genes, spots) and n columns are candidate friends (e.g. samples, cell types).

For each row i we have a vector of column ranks

$$r_1 \le r_2 \le \cdots \le r_k, \quad r_j \in \{1, \dots, m\},$$

where $k = n$ and the ranks are produced by `row.int.ranks()` (independent column-wise ranking, ties broken at random).

The **step (bi-uniform) model** posits that the k ranks come from a mixture of two uniform distributions:

- the k_1 "friend" ranks are i.i.d. Uniform$\{1, \dots, \ell_1\}$,
- the remaining $k - k_1$ "non-friend" ranks are i.i.d. Uniform$\{\ell_1 + 1, \dots, m\}$,

for some split rank $\ell_1 \in \{1, \dots, m-1\}$.

---

2. The log-likelihood

Given the split rank ℓ_1, the number of friends implied by the data is

$$k_1(\ell_1) = \#\{j : r_j \le \ell_1\}.$$

Setting $p_1 = k_1/k$, the log-likelihood of the step model is

$$\mathcal{L}(\ell_1) = k_1 \log\frac{p_1}{\ell_1} + (k - k_1)\log\frac{1 - p_1}{m - \ell_1} = -k_1 \log \ell_1 - (k - k_1)\log(m - \ell_1) + C(k_1),$$

where $C(k_1) = k_1 \log(k_1/k) + (k - k_1)\log((k - k_1)/k)$ depends only on k_1, not on ℓ_1.

The **null (uniform) model** has $\ell_1 = m$ (no step):

$$\mathcal{L}_0 = k \log\frac{1}{m}.$$

---

3. The original algorithm — $\mathbf{O}(m)$ per row

The original implementation (`step.fit.ln.likelihoods`) iterates over every integer $\ell_1 \in \{1, \ldots, m-1\}$, maintaining $k_1(\ell_1)$ with a running counter. Total cost per row: $\mathbf{O}(\boldsymbol{m})$. Total cost for the full matrix: $\mathbf{O}(\boldsymbol{m} \times \boldsymbol{n} \times \boldsymbol{m}) = \mathbf{O}(\boldsymbol{m^2 n})$.

For a typical VisiumHD 8 µm dataset ($m \approx 600\,000$, $n = 20$):

$$O(m^2 n) \approx 6 \times 10^{11} \text{ operations} \quad \Rightarrow \quad \approx 6 \text{ hours.}$$

---

4. Key mathematical observation

For a **fixed** k_1 the function $\ell_1 \mapsto \mathcal{L}(\ell_1)$ (with k_1 treated as constant, i.e. ignoring that k_1 jumps at rank values) is

$$f(\ell_1) = -k_1 \log \ell_1 - (k - k_1) \log(m - \ell_1) + C(k_1).$$

Second derivative:

$$f''(\ell_1) = \frac{k_1}{\ell_1^2} + \frac{k - k_1}{(m - \ell_1)^2} > 0 \quad \text{for } 0 < k_1 < k.$$

Therefore f is **strictly convex** in ℓ_1. In particular:

> **Lemma.** Over any closed interval $[\ell_{\min}, \ell_{\max}]$, the maximum of f is attained at one of the two endpoints $\ell_{\min}$ or $\ell_{\max}$.

The unique minimum of f is at

$$\ell_1^* = \frac{k_1\, m}{k},$$

which lies strictly inside the interval for generic data.

---

5. The valid interval for each k_1

Because the ranks are integers, $k_1(\ell_1) = k_1$ if and only if

$$r_{k_1} \le \ell_1 \le r_{k_1+1} - 1,$$

where $r_1 \le \cdots \le r_k$ are the **sorted** ranks of the row.

Define:

$$\ell_{k_1}^- = r_{k_1}, \qquad \ell_{k_1}^+ = r_{k_1+1} - 1.$$

The interval is **non-empty** iff $r_{k_1} < r_{k_1+1}$ (no tie between the k_1-th and (k_1+1)-th smallest ranks). When $r_{k_1} = r_{k_1+1}$ the split "at k_1 friends" is not achievable and is skipped.

The intervals for $k_1 = 1, 2, \ldots, k-1$ are **non-overlapping** and cover $\{r_1, \ldots, r_k - 1\}$ without gaps.

---

6. The optimized algorithm — O(k) per row

By the Lemma, for each valid k_1 we only need to evaluate $\mathcal{L}$ at the two boundary points $\ell^-_{k_1}$ and $\ell^+_{k_1}$:

```
for k1 in 1 … k-1:
    l_min ← r[k1]
    l_max ← r[k1+1] - 1
    if l_min > l_max: skip          # tied adjacent ranks
    evaluate L(l_min) and L(l_max)
    keep the larger value (prefer l_max on exact tie,
                           consistent with historical convention)

best_split ← argmax over k1 of the stored maximum
             (ties resolved by largest l1, then largest k1)
```

Total evaluations per row: at most $2(k-1)$ — $\mathbf{O(k) = O(n)}$.
Total cost for the full matrix: $\mathbf{O(m \times n \times k) = O(m \times n^2)}$.

For $m \gg n$ (which is the case in all practical datasets) this is an improvement of factor m/n:

Dataset	m	n	Old O($m^2 n$)	New O(mn^2)	Speedup
GSE112026 RNAseq	20 531	72	~2.5 min	~1.5 s	~100×
VisiumHD CRC 8 µm	600 000	20	~6 h	~50 s	~430×
Benchmark (random)	100 000	20	~68 min	~8 s	**490×**

---

7. Tie-breaking convention

When two different values of k_1 yield the same maximum log-likelihood, the implementation selects the split with the **largest** ℓ_1 (i.e. the largest threshold rank), consistent with the original algorithm's `max(which(ll == max_ll))` rule. Because the valid intervals are ordered — $\ell^+_{k_1} < \ell^-_{k_1+1}$ — this is equivalent to choosing the largest k_1 among tied splits, with the additional tie-break of preferring $\ell^+_{k_1}$ over $\ell^-_{k_1}$ within the same k_1.

---

8. Correctness guarantee

The two algorithms are **exactly equivalent** (not merely approximate):

- For each valid k_1, the original algorithm evaluates $\mathcal{L}(\ell_1)$ at every integer $\ell_1 \in [\ell_{k_1}^-, \ell_{k_1}^+]$. By the Lemma the maximum is always at one of the two endpoints.
- The new algorithm evaluates only those two endpoints and applies the same tie-breaking.
- Therefore both algorithms return the same optimal (k_1^*, ℓ_1^*) pair.

Empirical verification: both algorithms produce byte-identical results on the GSE112026 dataset (20 531 × 72) and on random matrices up to 100 000 × 20 (all 34 unit tests pass on both branches).

---

9. Implementation

Internal function `.step_fit_compact(ranks, max.possible.rank)` in `R/step.fit.ln.likelihoods.r`. Not exported; called by `best.step.fit()` and `best.step.fit.bic()`. The public API of `step.fit.ln.likelihoods()` is unchanged.

```
.step_fit_compact <- function(ranks, max.possible.rank) {
    columns.order <- order(ranks)
    sorted_ranks  <- ranks[columns.order]   # r_1  r_2  …  r_k
    k <- length(sorted_ranks)

    best_ll_by_k1 <- rep(-Inf, k)
    best_l1_by_k1 <- integer(k)

    for (k1 in seq_len(k - 1L)) {
        l1_min <- sorted_ranks[k1]
        l1_max <- sorted_ranks[k1 + 1L] - 1L
        if (l1_min > l1_max) next          # tied adjacent ranks

        p1      <- k1 / k
        log_p1  <- log(p1)
        log_1p1 <- log(1 - p1)

        ll_min <- k1 * (log_p1 - log(l1_min)) +
                  (k - k1) * (log_1p1 - log(max.possible.rank - l1_min))

        if (l1_min == l1_max) {
            best_l1_by_k1[k1] <- l1_min
            best_ll_by_k1[k1] <- ll_min
        } else {
            ll_max <- k1 * (log_p1 - log(l1_max)) +
```

```
                    (k - k1) * (log_1p1 - log(max.possible.rank - l1_max))
            if (ll_max >= ll_min) {
                best_l1_by_k1[k1] <- l1_max
                best_ll_by_k1[k1] <- ll_max
            } else {
                best_l1_by_k1[k1] <- l1_min
                best_ll_by_k1[k1] <- ll_min
            }
        }
    }

    list(
        columns.order = columns.order,
        best_ll_by_k1 = best_ll_by_k1,
        best_l1_by_k1 = best_l1_by_k1,
        uniform_ll    = k * log(1 / max.possible.rank)
    )
}
```

---

10. Open questions for discussion

1. **Exact vs. approximate.** The proof relies on the fact that $\mathcal{L}(\ell_1)$ is strictly convex in ℓ_1 *between* rank boundaries (where k_1 is constant). Could there be a pathological input where floating-point cancellation causes the computed $f(\ell_{\min})$ or $f(\ell_{\max})$ to be slightly inaccurate relative to an interior integer? In practice, both arguments of `log` are well away from 0 for any real dataset ($\ell_1 \geq 1$, $m - \ell_1 \geq 1$), so this is not a concern.

2. **Alternative: vectorized O(m) with `findInterval`.** An intermediate approach would keep the full $\ell_1 \in \{1, \ldots, m-1\}$ enumeration but replace the R loop with a vectorized `findInterval` call. This would still be O(m) per row but potentially 10–50× faster than the loop. The O(k) approach is preferred because it is asymptotically better and already implemented.

3. **Parallelism.** The O(k) inner loop per row makes each row extremely cheap (~14 µs on Apple M-series for $k = 20$). At this granularity, BiocParallel worker overhead (≈ 1 ms per task) dominates. For very large m, row-level parallelism should batch rows rather than submit them one by one — a possible future improvement.